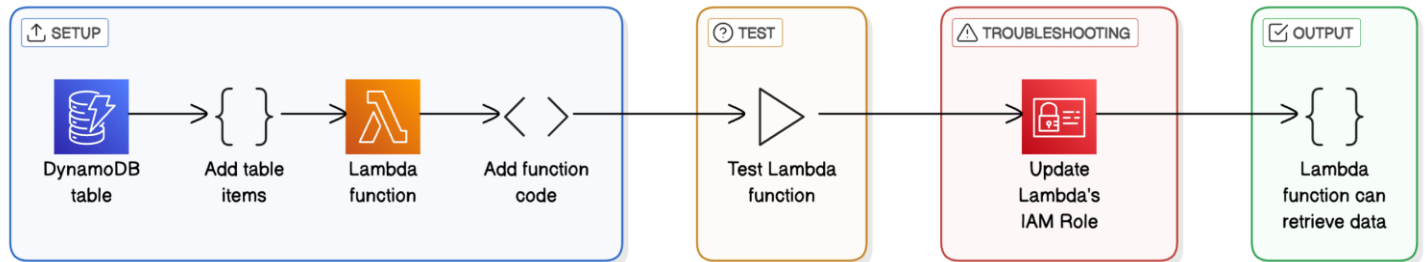


FETCH DATA WITH AWS LAMBDA - STEP-BY-STEP

DOCUMENTATION



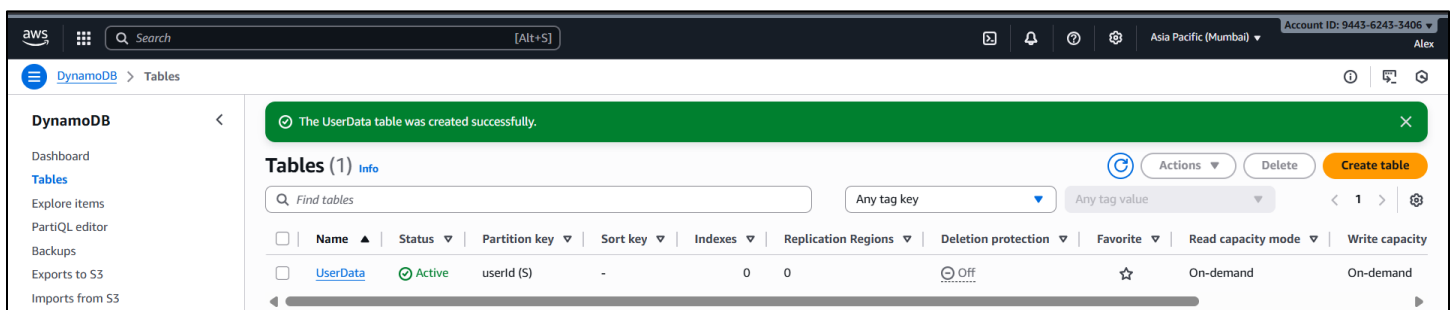
This documentation provides a step-by-step process for creating a DynamoDB table and a serverless AWS Lambda function that retrieves data from the table. It covers table creation, inserting sample data, Lambda implementation, testing, permissions, and cleanup. This project forms the data tier of a three-tier architecture solution.

Step 1: Set up DynamoDB

Create a DynamoDB table to store user data.

Actions:

- Log in to the AWS Management Console.
- Navigate to DynamoDB in the selected AWS region.
- Click 'Create table'.
- For Table name, enter: UserData.
- For Partition key, enter: userId (String).
- Leave other settings as default and select 'Create table'.



Step 2: Add a Table Item

Insert a sample item into the UserData table so Lambda has data to retrieve.

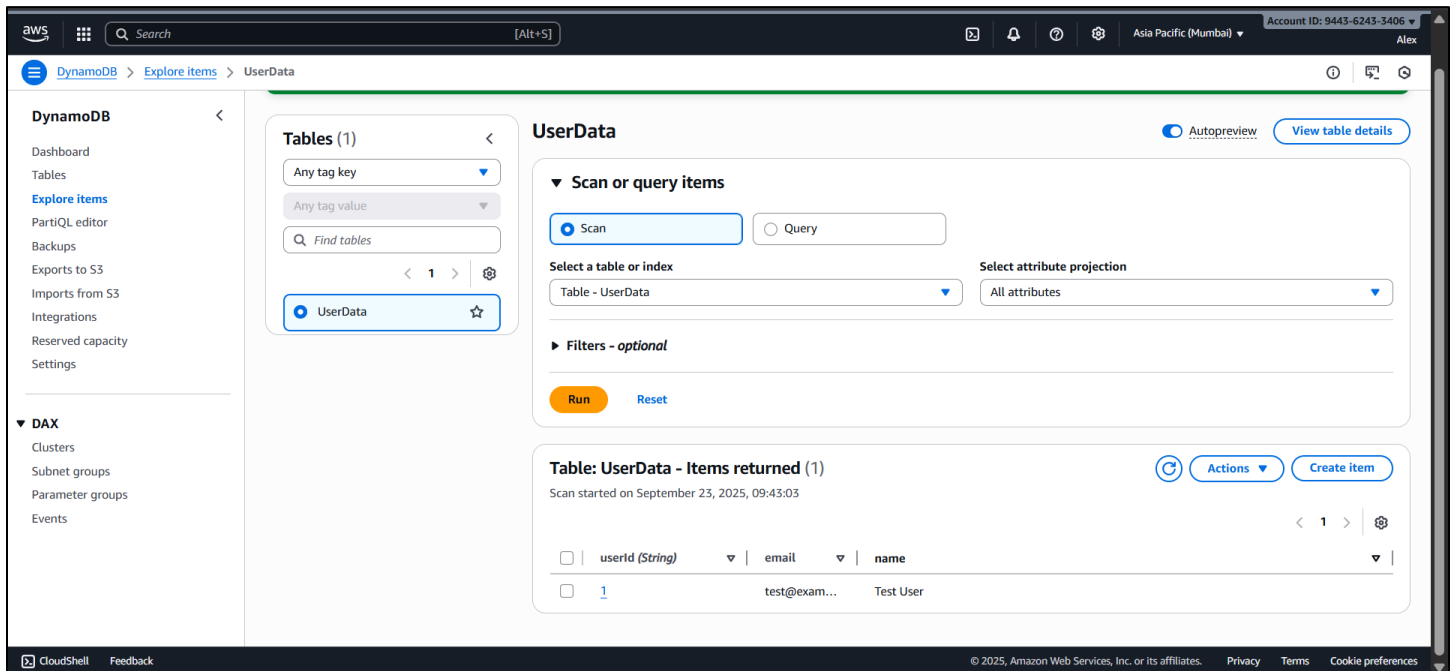
Actions:

- Wait for the table status to become 'Active'.
- Open the UserData table and select 'Explore table items'.
- Click 'Create item' and switch to JSON view.

- Paste the following JSON and create the item:

```
{
  "userId": "1",
  "name": "Test User",
  "email": "test@example.com"
}
```

- Verify the item appears in the Items list.

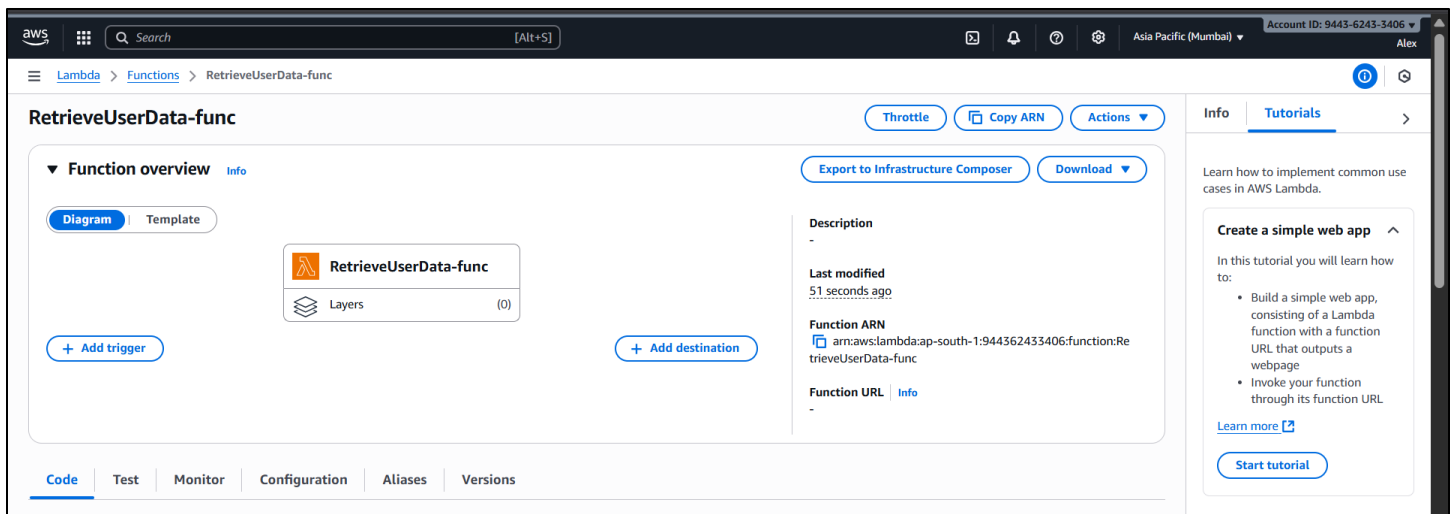


Step 3: Create the Lambda Function

Create an AWS Lambda function that will read data from DynamoDB.

Actions:

- Navigate to the Lambda service in the AWS Console.
- Click 'Create function' and choose 'Author from scratch'.
- For Function name, enter: RetrieveUserData.
- Select a Node.js runtime and keep 'Create a new role with basic Lambda permissions'.
- Click 'Create function'.



Step 4: Implement the Lambda Function Logic

Add code to the Lambda function to retrieve items from DynamoDB and deploy it.

Actions:

- In the Lambda console, open the Code source panel.
- Replace the default code with the provided Node.js code (remember to set your region):

```
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
```

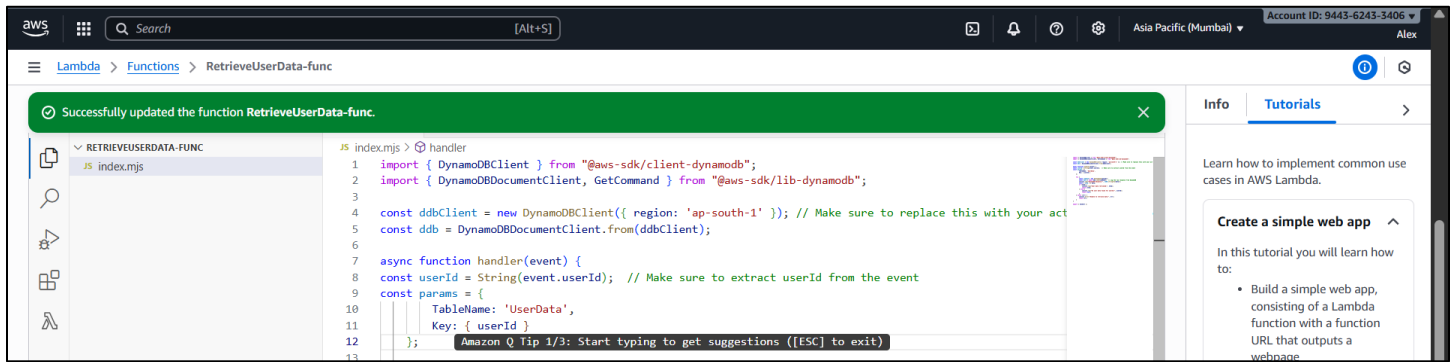
```
const ddbClient = new DynamoDBClient({ region: 'enter your region' });
const ddb = DynamoDBDocumentClient.from(ddbClient);
```

```
async function handler(event) {
  const userId = String(event.userId);
  const params = { TableName: 'UserData', Key: { userId } };
```

```
  try {
    const command = new GetCommand(params);
    const data = await ddb.send(command);
    const { Item } = data;
    if (Item) return Item;
    return null;
  } catch (err) {
    console.error('Unable to retrieve data:', err);
    return err;
  }
}
```

```
export { handler };
```

- Click 'Deploy' to save and publish your code.



Step 5: Write a Lambda Function Test

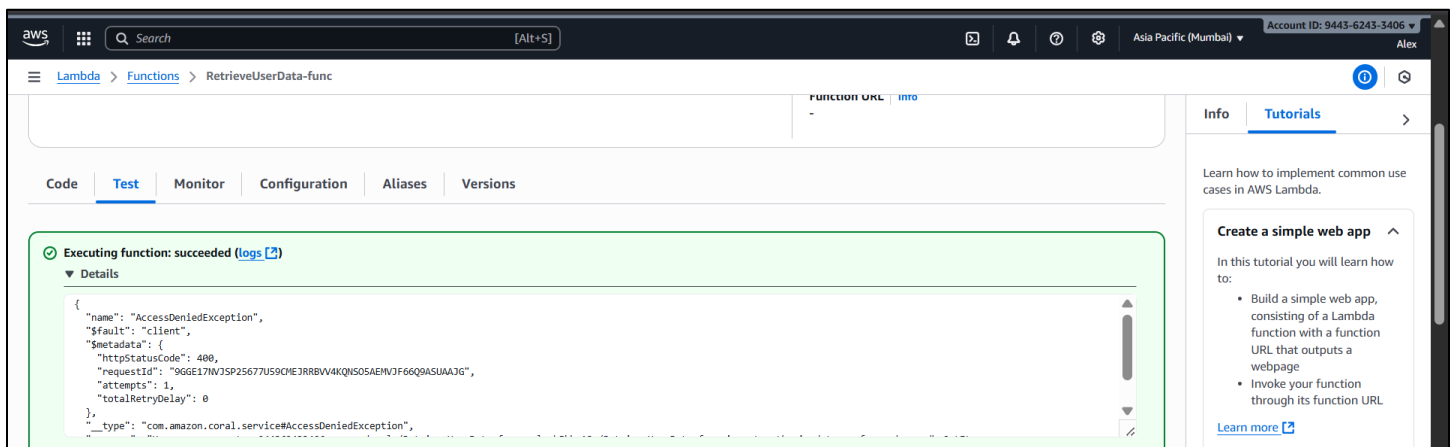
Create and run a test event to validate the Lambda function's behavior.

Actions:

- In the Lambda console, open the Test tab.
- Create a new test event with the JSON:

```
{
  "userId": "1"
}
```

- Run the test and inspect the execution results and logs.
- If you see an AccessDenied error, proceed to Step 6 to update permissions.

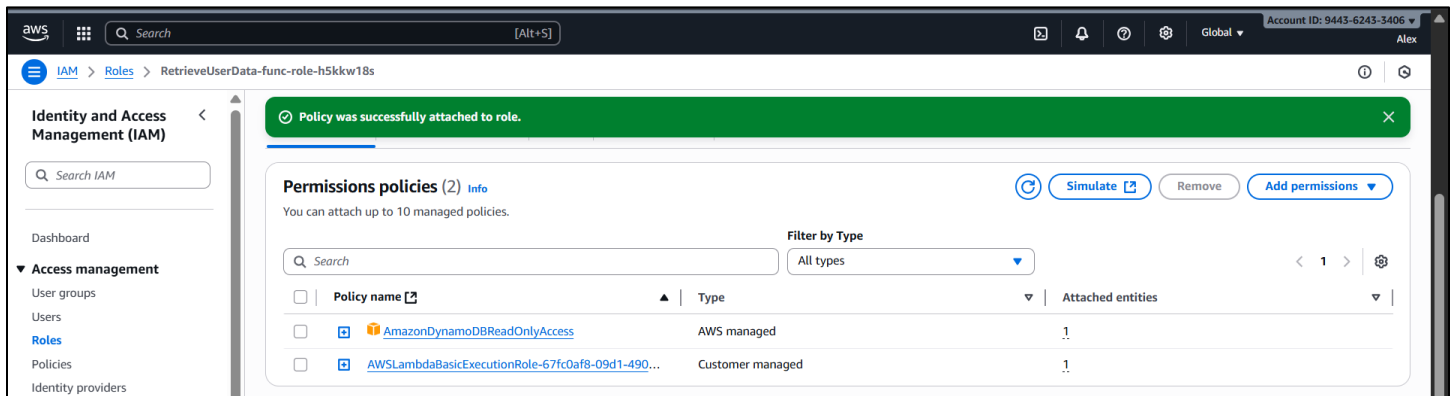


Step 6: Grant DynamoDB Access to Lambda

Attach a read-only DynamoDB policy to the Lambda execution role so it can read table items.

Actions:

- In the Lambda console, go to the Configuration tab and select Permissions.
- Click the execution role to open the IAM console.
- Select 'Add permissions' -> 'Attach policies'.
- Search for and attach the policy: AmazonDynamoDBReadOnlyAccess.

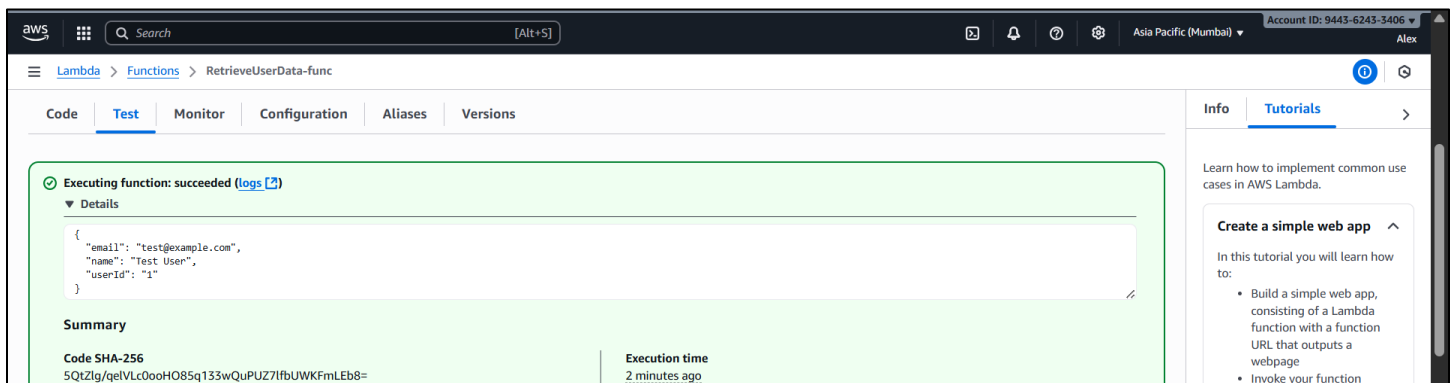


Step 7: Re-test Your Lambda Function

Re-run the Lambda test to ensure the function retrieves data successfully.

Actions:

- Open the Test tab and run the previously created test event.
- Verify the response contains the user item from DynamoDB.
- Inspect logs for any errors and confirm successful execution.



Step 8: Secret Mission — Tighten DynamoDB Security

Harden permissions by creating a more restrictive inline or custom policy for the Lambda role.

Actions:

- Instead of broad read-only policies, create a custom IAM policy allowing s3:GetItem or dynamodb:GetItem specifically for the UserData table.
- Attach the custom policy to the Lambda execution role.
- Re-run tests to confirm functionality.

User groups
Users
Roles
Policies
Identity providers
Account settings
Root access management [New](#)
▼ Access reports
Access Analyzer

DynamoDBGetItemPolicy

Copy JSON

Edit

```
1- {  
2-   "Version": "2012-10-17",  
3-   "Statement": [  
4-     {  
5-       "Sid": "VisualEditor0",  
6-       "Effect": "Allow",  
7-       "Action": "dynamodb:GetItem",  
8-       "Resource": "arn:aws:dynamodb:ap-south-1:944362433406:table/UserData"  
9-     }  
10-  ]  
11- }
```

✓ Executing function: succeeded ([logs](#))
▼ Details

```
{  
  "email": "test@example.com",  
  "name": "Test User",  
  "userId": "1"  
}
```

Summary

Code SHA-256

Execution time

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

^

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

Conclusion

You've learned how to create a DynamoDB table, insert sample data, implement a Lambda function to retrieve that data, grant appropriate permissions, and test the function. This completes the data tier for the three-tier architecture series.